Studio Nx

백엔드 227 — 트랙 안 장치 모델

2026-10-18

Backend(FastAPI/Python) 227 — `TrackInstrument` · `TrackPlugin` · `TrackPluginLink`

항목	내용
파일	`studionx/backend/app/models/track_plugin.py`
작업	S1-09 기본 악기·이펙트: 트랙 디바이스 표 세 개 (PRD 02 FR-245, FR-263, FR-267, FR-269)
작성	Claude (AI), 2026-10-18, 사용자 검토
이전 · 다음	226 `Plugin` · 228 `Track` Phase 2 칸

1. 하는 일

트랙 한 줄의 **소리 만들기 구성**을 담는 표 세 개입니다. 정의(226 `plugins`)와 실제로 트랙에 붙은 칸을 나눠 두었습니다.

표	넣은 것	뜻
`track_instruments`	`track_id`, `plugin_id`, `slot_index`(기본 0), `params`, `volume_db`, `pan`, `is_solo`, `is_enabled`	트랙의 악기 한 칸. `params` 에 그 악기의 파라미터가 들어갑니다 (FR-245·264)
`track_plugins`	위와 같고 `volume_db`·`pan`·`is_solo` 대신 `node_x`, `node_y`, `is_bypassed`	트랙의 이펙트 한 칸. `slot_index` 가 체인 순서, `node_x`·`node_y` 는 갈래 화면의 카드 자리입니다 (FR-263·269)
`track_plugin_links`	`track_id`, `from_track_plugin_id`, `from_port`, `to_track_plugin_id`, `to_port`, `gain_db`	갈래를 나누고 합치는 연결선입니다. `from` 하나에서 여럿으로 가면 갈래, 여럿이 `to` 하나로 모이면 병합입니다 (FR-267)

`from_track_plugin_id` 나 `to_track_plugin_id` 가 비어 있으면(NULL) 각각 트랙 입력(소스)과 트랙 출력을 뜻합니다.

2. 언제 쓰나

쓰는 곳	
`track_service.list_instruments` · `add_instrument` · `update_instrument` · `remove_instrument` (234)	악기 칸을 읽고 붙이고 고치고 뗍니다
`track_service.list_effects` · `add_effect` · `update_effect` · `remove_effect` (235)	이펙트 칸을 다룹니다
`track_service.list_links` · `add_link` · `remove_link` (236)	연결선을 다룹니다
`Track.instruments` · `Track.plugins` · `Track.plugin_links` 관계 (228)	트랙을 지울 때 DB CASCADE 로 함께 지워집니다

3. 코드

`TrackInstrument`

```
    track_id: Mapped[int] = mapped_column(BigInteger, ForeignKey("tracks.track_id", ondelete="CASCADE"))
    plugin_id: Mapped[int] = mapped_column(INTEGER(unsigned=True), ForeignKey("plugins.plugin_id"))
    slot_index: Mapped[int] = mapped_column(TINYINT(unsigned=True), server_default=text("0"))
    params: Mapped[dict[str, Any] | None] = mapped_column(JSON)
    volume_db: Mapped[Decimal] = mapped_column(Numeric(5, 2), server_default=text("0.00"))
    pan: Mapped[Decimal] = mapped_column(Numeric(3, 2), server_default=text("0.00"))
    is_solo: Mapped[bool] = mapped_column(Boolean, server_default=text("0"))
    is_enabled: Mapped[bool] = mapped_column(Boolean, server_default=text("1"))
```

`TrackPlugin`

```
    slot_index: Mapped[int] = mapped_column(TINYINT(unsigned=True))          # 체인 순서 (필수)
    node_x: Mapped[int | None] = mapped_column(INTEGER)                      # 갈래 화면 자리
    node_y: Mapped[int | None] = mapped_column(INTEGER)
    params: Mapped[dict[str, Any]] = mapped_column(JSON)
    is_bypassed: Mapped[bool] = mapped_column(Boolean, server_default=text("0"))
```

`TrackPluginLink`

```
    track_id: Mapped[int] = mapped_column(BigInteger, ForeignKey("tracks.track_id", ondelete="CASCADE"))
    from_track_plugin_id: Mapped[int | None] = mapped_column(
        BigInteger, ForeignKey("track_plugins.track_plugin_id", ondelete="CASCADE")
    )
    from_port: Mapped[int] = mapped_column(TINYINT(unsigned=True), server_default=text("0"))
    to_track_plugin_id: Mapped[int | None] = mapped_column(
        BigInteger, ForeignKey("track_plugins.track_plugin_id", ondelete="CASCADE")
    )
    to_port: Mapped[int] = mapped_column(TINYINT(unsigned=True), server_default=text("0"))
    gain_db: Mapped[Decimal] = mapped_column(Numeric(5, 2), server_default=text("0.00"))
```

문법 풀이

식	뜻
`ondelete="CASCADE"`	부모(트랙, 이펙트)를 지우면 이 행도 DB 가 함께 지웁니다
`ForeignKey("plugins.plugin_id")` (ON DELETE 없음)	쓰이던 플러그인은 지울 수 없게 DB 가 막습니다. 실수로 소리가 사라지는 일을 막습니다
`TINYINT(unsigned=True)`	0~255 정수입니다. 칸 번호와 포트 번호는 음수가 될 수 없습니다
`Mapped[int | None]`	비어 있을 수 있는 칸입니다. `node_x`·`from`·`to` 가 그렇습니다
`server_default=text("0")`	값을 안 주면 DB 가 채웁니다. 응답에는 실제 값이 담겨 나옵니다
`JSON`	파라미터 묶음을 그대로 담습니다. 플러그인마다 파라미터가 달라도 표를 바꾸지 않습니다

4. 왜 이렇게

선택	이유
악기와 이펙트를 다른 표로	악기는 트랙에 하나 이상 겹쳐 놓고(레이어) 볼륨·팬·솔로를 따로 가집니다. 이펙트는 체인 순서와 갈래 자리를 가집니다. 담는 칸이 달라 표를 나눴습니다
빈 칸 대신 `NULL` 로 시작·끝	연결선을 한 표로 다루면서 트랙 입력과 출력을 함께 그릴 수 있습니다 (FR-267·269)

선택	이유
연결선을 따로 저장	갈래는 한 이펙트에서 여럿으로 나가고 다시 합쳐집니다. 나무 구조는 표 한 줄로 담을 수 없습니다 (FR-265·267)
칸 번호에 UNIQUE 를 둠	같은 트랙의 같은 칸에 두 행이 들어가면 화면 순서를 정할 수 없습니다. DB 가 `UNIQUE (track_id, slot_index)` 로 막고, 서비스가 먼저 확인해 400 으로 알려 줍니다 (234 §3)
`passive_deletes=True`	삭제는 DB CASCADE 에 맡깁니다. 앱이 자식 행을 하나씩 지우다 빠뜨릴 일이 없습니다

5. 시험

`tests/test_plugin_model.py` 에서 세 표의 칸을 실제 MySQL 과 비교하고, 같은 칸에 두 번 넣으면 `IntegrityError` 가 나는지, 트랙을 지우면 세 표의 행이 모두 사라지는지 확인합니다.

> 트랙을 지운 뒤 앱이 들고 있던 관계 목록(`track.instruments`)에는 지워진 행이 남을 수 있습니다. 그래서 목록 API 는 관계 대신 쿼리로 다시 읽습니다 (234 참고).